

Set Up Your Own Cost Alerts

Three copy-paste alerts so you find out about your serverless spend *before* the platform team has to ping you

Circle K · Data & Analytics

Audience: notebook users · v1.0 · May 2026

WHY BOTHER

The platform team watches the top serverless spenders across the account. If you cross their thresholds, you'll hear from Sairam. These three personal alerts give you the same information *first* — and quietly, by email — so you can react before it becomes a conversation. Setup takes about 10 minutes per alert. They run forever after that. You'll thank yourself the day you forget a notebook tab on a Friday afternoon.

1 · Before you start

- You need access to **system tables**. Test it: open a SQL editor and run `SELECT current_user(), COUNT(*) FROM system.billing.usage LIMIT 1`. If it works, you're set. If not, ask Sairam.
- You need a **SQL warehouse** available to run the alert query on. A small serverless warehouse is fine — alert queries are cheap.
- Every alert below filters on `current_user()`. **You don't need to edit your email anywhere** — when the alert runs, Databricks substitutes your identity automatically because the alert runs as you (the owner).

2 · Alert 1 — Tell me when I spent more than \$X yesterday

SCHEDULE: DAILY 07:00

LAG: ~12 H (BILLING TABLE)

SEND TO: YOURSELF

The query

Triggers if your total serverless DBU spend yesterday exceeded the threshold. **Edit one number** — your daily threshold in dollars. CK starting point: **\$100** for analysts, **\$300** for heavy users.

```
WITH yesterday AS (  
  SELECT  
    SUM(u.usage_quantity * lp.pricing.default) AS list_cost,  
    SUM(u.usage_quantity)                      AS dbus  
  FROM system.billing.usage u  
  INNER JOIN system.billing.list_prices lp  
    ON u.cloud = lp.cloud  
    AND u.sku_name = lp.sku_name  
    AND u.usage_start_time >= lp.price_start_time  
    AND (u.usage_end_time <= lp.price_end_time OR lp.price_end_time IS NULL)  
  WHERE u.usage_date = CURRENT_DATE() - INTERVAL 1 DAY  
    AND u.sku_name LIKE '%SERVERLESS%'  
    AND u.identity_metadata.run_as = current_user()  
)  
SELECT ROUND(list_cost, 2) AS yesterday_dollars,  
       ROUND(dbus, 2)     AS yesterday_dbus  
FROM yesterday  
WHERE list_cost > 100;  -- <-- change this number
```

Setting up the alert

- 1 SQL → Alerts → Create alert. Give it a name like *My serverless spend > \$100/day*.
- 2 Paste the query above. Pick a SQL warehouse to run on.
- 3 Trigger condition: `QUERY_RESULT_ROWS > 0` (any row triggers).
- 4 Schedule: *Daily at 07:00*.
- 5 Destination: *Email me*.
- 6 Subject: [Cost] My serverless spend yesterday: `${{QUERY_RESULT_TABLE}}`
- 7 Save. Test by lowering the threshold to `$0` for a day, then put it back.

3 · Alert 2 — Tell me when one of my queries has been running too long

SCHEDULE: EVERY 15 MIN

LAG: 1-2 MIN (QUERY HISTORY)

SEND TO: YOURSELF

The query

Triggers when any SQL statement *you* started is still running after the runtime threshold. **Edit one number** — the hours threshold. Starting point: **1 hour**.

```
SELECT
  qh.statement_id,
  qh.start_time,
  ROUND(qh.total_duration_ms / 1000.0 / 3600.0, 2) AS runtime_hours,
  LEFT(qh.statement_text, 200) AS statement_preview
FROM system.query.history qh
WHERE qh.executed_by = current_user()
  AND qh.start_time >= CURRENT_TIMESTAMP() - INTERVAL 24 HOURS
  AND qh.end_time IS NULL
  AND qh.total_duration_ms > 1 * 3600 * 1000 -- <-- change "1" to your hours
ORDER BY runtime_hours DESC;
```

Setting up the alert

- 1 SQL → Alerts → Create alert. Name: *My query running > 1h*.
- 2 Paste the query. Pick a SQL warehouse.
- 3 Trigger condition: `QUERY_RESULT_ROWS > 0`.
- 4 Schedule: *Every 15 minutes*.
- 5 Destination: *Email me*.
- 6 In the alert body, drop in `{{QUERY_RESULT_TABLE}}` so the email tells you which statement.
- 7 Save.

IF THIS ALERT FIRES

Open SQL → *Query History*, paste the `statement_id` into the filter, and click **Cancel** in the top right. Costs stop within seconds. If you closed the notebook tab thinking that killed the query — it doesn't. The serverless backend keeps running the statement until it either finishes or you cancel it (or hits the 2.5 h default timeout).

4 · Alert 3 — Tell me if my spend doubled this week

SCHEDULE: WEEKLY MONDAY 07:00

LAG: ~24 H

SEND TO: YOURSELF

The query

Triggers if your serverless spend in the last 7 days is more than 2× the prior 7 days. Catches drift you wouldn't notice day to day. **No editing needed** — multiplier and lookback are baked in (change them if you want to be more or less sensitive).

```
WITH base AS (
  SELECT
    CASE
      WHEN u.usage_date >= CURRENT_DATE() - INTERVAL 7 DAYS THEN 'this_week'
      WHEN u.usage_date >= CURRENT_DATE() - INTERVAL 14 DAYS THEN 'prior_week'
    END
    SUM(u.usage_quantity * lp.pricing.default) AS list_cost
  FROM system.billing.usage u
  INNER JOIN system.billing.list_prices lp
    ON u.cloud = lp.cloud
    AND u.sku_name = lp.sku_name
    AND u.usage_start_time >= lp.price_start_time
    AND (u.usage_end_time <= lp.price_end_time OR lp.price_end_time IS NULL)
  WHERE u.usage_date >= CURRENT_DATE() - INTERVAL 14 DAYS
    AND u.sku_name LIKE '%SERVERLESS%'
    AND u.identity_metadata.run_as = current_user()
  GROUP BY bucket
),
pivot AS (
  SELECT
    MAX(CASE WHEN bucket = 'this_week' THEN list_cost END) AS this_week_dollars,
    MAX(CASE WHEN bucket = 'prior_week' THEN list_cost END) AS prior_week_dollars
  FROM base
)
SELECT
  ROUND(this_week_dollars, 2) AS this_week_dollars,
  ROUND(prior_week_dollars, 2) AS prior_week_dollars,
  ROUND(this_week_dollars / NULLIF(prior_week_dollars, 0), 2) AS multiplier
FROM pivot
WHERE prior_week_dollars > 25 -- ignore tiny baselines
  AND this_week_dollars > 2 * prior_week_dollars; -- 2x trigger
```

Setting up the alert

- 1 SQL → Alerts → Create alert. Name: *My spend doubled this week*.
- 2 Paste the query.
- 3 Trigger condition: `QUERY_RESULT_ROWS > 0`.
- 4 Schedule: *Weekly, Monday 07:00*.
- 5 Destination: *Email me*. Body: include `{{QUERY_RESULT_TABLE}}`.

5 · When an alert fires — your 60-second playbook

Alert	What to do
Daily spend > \$X	Open Genie Code in agent mode and ask: " <i>Which notebooks did I run yesterday that incurred the most cost?</i> " Same trick Megan used in her 5/20 walkthrough. Identify the offender → either rewrite the query (use the runbook in §6) or accept it as expected.
My query running > 1h	Open <i>SQL</i> → <i>Query History</i> . Find the <code>statement_id</code> from the email. If unintended → Cancel button. If intended → consider whether you should have run <code>EXPLAIN FORMATTED</code> first (see runbook §5).
Spend doubled WoW	Likely a new workstream or a missed cache. Compare your week-on-week activity in Genie Code. If it's a single notebook driving it, that's your refactor candidate.

COMPANION: SERVERLESS NOTEBOOK RUNBOOK

For *how* to fix the underlying behaviors (the do/don't pairs, partition pruning, `.cache()`, query profiles), see the developer-facing runbook in this same folder. These alerts tell you *when*; the runbook tells you *what to change*.

6 · For team leads — alerts on your whole team

The recipes above scope to a single user. To watch a team, swap the `identity_metadata.run_as` filter for one of these patterns.

6.1 By email domain or list

```
WHERE u.identity_metadata.run_as IN (
  'alice@circlek.com',
  'bob@circlek.com',
  'carol@circlek.com'
)
-- or by domain:
WHERE u.identity_metadata.run_as LIKE '%@circlekeurope.com'
```

6.2 By custom tag (recommended once tagging is enforced)

If your team's notebooks and jobs are tagged with a `team` or `cost_center` custom tag (via a serverless usage policy), filter on it. This is more reliable than maintaining an email list.

```
WHERE u.custom_tags.team = 'merchandising-analytics'
-- or
WHERE u.custom_tags.cost_center = 'ck-na-supply-chain'
```

6.3 Recommended team alerts

Alert	Suggested threshold	Cadence
Team daily serverless spend	\$1,000 (analysts), \$3,000 (DE teams)	Daily 07:00
Any team member's daily spend	\$300	Daily 07:00
Team 30-day spend	Quarterly budget ÷ 3	Daily
Untagged team activity	any (returns rows = something is untagged)	Weekly

DON'T PILE EVERY TEAM MEMBER ONTO THE DESTINATION LIST

Send team-level alerts to **one** destination (yourself, a team DL, or a Teams channel) — not every individual. Individual alerts in §2-4 cover the personal-accountability side. Team alerts are about *you* seeing patterns, not paging the whole team every morning.

7 · Where to look beyond alerts

- **Genie Code in agent mode** — ask "Tell me about my consumption on serverless notebooks" or "Which notebook did I run on {date} that drove the cost?" Works on natural language; runs against your own system tables.
- **SQL → Query History** — your last 30 days of statements, with cost and bytes-scanned columns.
- **Account Usage Dashboard** (if imported by your workspace admin) — pre-built spend breakdowns by user, workspace, product.
- **Companion: Serverless Notebook Runbook** — the developer habits that prevent these alerts from firing in the first place.